

RW蓝牙小程序协议文档

1. 微信小程序蓝牙接口介绍

微信小程序蓝牙开发请参考官方文档, 具体不再详细说明.

小程序蓝牙指导文档: <https://developers.weixin.qq.com/miniprogram/dev/framework/device/ble.html>

小程序蓝牙API文档: <https://developers.weixin.qq.com/miniprogram/dev/api/device/bluetooth/wx.stopBluetoothDevicesDiscovery.html>

1.1 蓝牙扫描设备

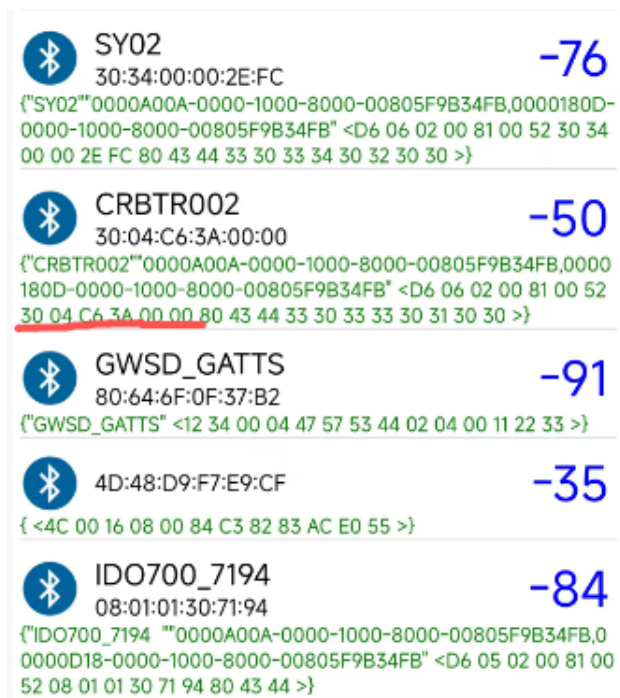
接口: `wx.startBluetoothDevicesDiscovery(Object object)`

API链接: <https://developers.weixin.qq.com/miniprogram/dev/api/device/bluetooth/wx.startBluetoothDevicesDiscovery.html>

搜索结果监听 `wx.onBluetoothDeviceFound(function listener)` 接口.

注意 `deviceId` 属性 Android为mac地址, iOS为udid, 如果iOS也想显示mac,可解析`advertisData`里的数据, 见下面图片标红内容.

扫描结果类似如下:



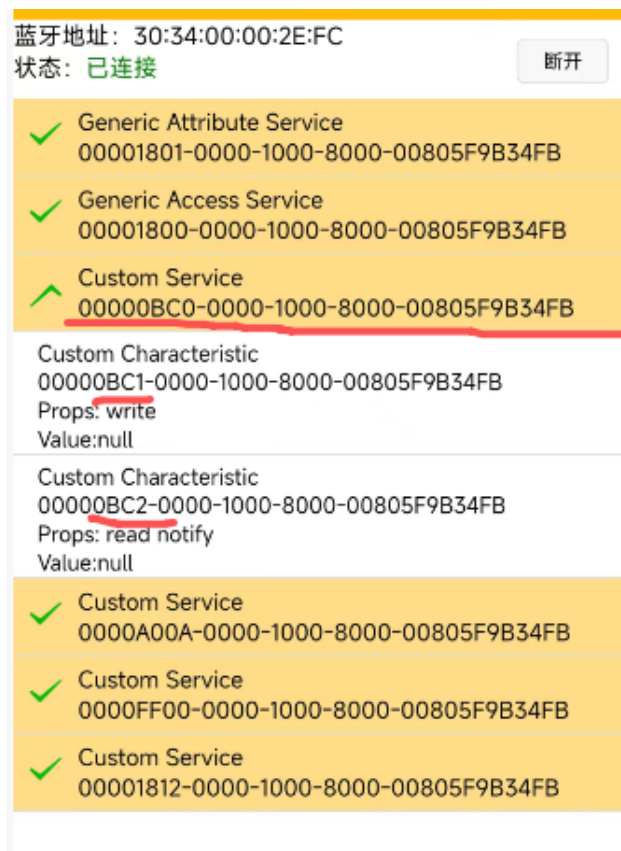
1.2 蓝牙连接, 服务和特征

(1)创建连接 `wx.createBLEConnection(Object object);`

(2)获取服务 `wx.getBLEDeviceServices(Object object);`

(3)获取特征 `wx.getBLEDeviceCharacteristics(Object object);`

注意: 戒指使用的服务为 OBC0, 写特征 OBC1, 接收特征 OBC2; 设备SY02从版本2.2.7支持小程序, 设备SY01从版本2.1.5支持小程序, 可先连接绑定RWFit app查看版本号信息确定, 如不支持可向厂家询问新版本升级。



1.3 蓝牙数据发送与接收

数据发送: `wx.writeBLECharacteristicValue(Object object)`

启用Notify特征可接收数据: `wx.notifyBLECharacteristicValueChange(Object object)`

数据接收监听: `wx.onBLECharacteristicValueChange(function listener)`

因写入的value为 ArrayBuffer类型数据,要注意转换. 比如同步时间指令,提供的十六进制字符串为: C601000413E802340001;

(1) 直接赋值为Uint8Array

```
// 固定赋值方式
const data = new Uint8Array([0xC6, 0x01, 0x00, 0x04, 0x13, 0xE8, 0x02, 0x34, 0x00, 0x01]);

// 转成 ArrayBuffer, 用于 wx.writeBLECharacteristicValue
const buffer = data.buffer;

wx.writeBLECharacteristicValue({
  deviceId,
  serviceId,
  characteristicId,
  value: buffer,
  success: res => console.log('write ok', res),
  fail: err => console.error('write fail', err),
```

```
});
```

(2) 把十六进制字符串转成 ArrayBuffer;

```
// 把十六进制字符串转成 ArrayBuffer (忽略空格、大小写)
function hexToArrayBuffer(hex) {
  if (!hex) return new ArrayBuffer(0);
  const clean = hex.replace(/\s+/g, '').toLowerCase();
  if (clean.length % 2 !== 0) throw new Error('HEX 长度必须为偶数');
  const len = clean.length / 2;
  const buf = new ArrayBuffer(len);
  const view = new Uint8Array(buf);
  for (let i = 0; i < len; i++) {
    view[i] = parseInt(clean.substr(i * 2, 2), 16);
  }
  return buf;
}

wx.writeBLECharacteristicValue({
  deviceId, serviceId, characteristicId,
  value: hexToArrayBuffer('C601000413E802340001'),
  success: res => console.log('write ok', res),
  fail: err => console.error('write fail', err),
});
```

数据接收监听处理例子:

```
wx.onBLECharacteristicValueChange(function (res) {
  const buffer = res.value;
  const parsed = parseBlePacket(buffer);
  console.log('解析结果:', parsed);
});

function parseBlePacket(buffer) {
  const dataView = new DataView(buffer);
  const totalLen = dataView.byteLength;
  if (totalLen < 6) {
    console.warn('数据长度不足');
    return null;
  }

  // 顶层结构
  const header = dataView.getUint8(0);
  const flag = dataView.getUint8(1);
  const dataLen = dataView.getUint16(2, false); // 大端长度
  const crc = dataView.getUint16(4, false);

  if (header !== 0xC6) {
    console.warn('头字节错误:', header.toString(16));
    return null;
  }
}
```

```
// 解析 Data 部分
const dataBytes = new Uint8Array(buffer.slice(6));
if (dataBytes.length < 3) {
  console.warn('Data 区域长度不足');
  return null;
}

// --- Data 格式 ---
// | 2字节 CMD+Key | 1字节 KeyFlag | N字节 Value |
const cmdKey = (dataBytes[0] << 8) | dataBytes[1];
const keyFlag = dataBytes[2];
const value = dataBytes.slice(3);

//下面可按协议文档格式解析value.
if (cmdKey == 0x051a){ //今天步数

}
else if (cmdKey == 0x0302){ //登录指令返回

}

}

// 工具函数
function bytesToHex(bytes) {
  return Array.from(bytes)
    .map(b => b.toString(16).padStart(2, '0'))
    .join(' ');
}
```

备注: 可使用工具小程序 科名BLE 协助查看广播包, 服务UDID, 特征等, 也可用其它BLE工具小程序,原理一样.

2. 功能指令操作

协议格式:

1字节	1字节	2字节	2字节	N字节
0xC6	Protocol flag	Len	CRC	Data
固定头	1. 0x01 主动写 2. 0x11 回复	数据长度	CRC16	数据

Data数据部分格式:

1字节	1字节	1字节	Len-3字节
CMD	Key	Key flag	Value
功能cmd	功能key	0x00 : 改(写)操作 0x10 : 查(读)操作 0x20 : 增操作 0x30 : 删操作	具体Value内容

CRC是对Data数据部分进行计算得到的值, CRC16 C语言参考算法如下:

```
static unsigned short CRC_TABLE[] = {
    0x72ae,0x2cd6,0x3d6c,0x4ae1,0x6784,0x18be,0x4823,0x0029,
    0x01eb,0x26e9,0x41bb,0x5af1,0x6df1,0x1649,0x5f90,0x6952,
    0x0099,0x0f3e,0x390c,0x7e87,0x153c,0x12db,0x2ea6,0x0bb3,
    0x54de,0x1547,0x4db7,0x4d06,0x491c,0x440d,0x305e,0x0124,
    0x26a6,0x428b,0x66bb,0x6443,0x4dc8,0x074d,0x2d12,0x39b3,
    0x1e1f,0x3b25,0x1238,0x4509,0x767d,0x7a5a,0x5d03,0x701f,
    0x323b,0x4e45,0x7ff5,0x7f96,0x6bfc,0x63cb,0x1ad4,0x6e5d,
    0x0732,0x56ae,0x0bdb,0x301c,0x030a,0x6b89,0x260d,0x2213,
    0x5cfd,0x6b36,0x5878,0x4b40,0x22ee,0x2350,0x759a,0x0120,
    0x0ddc,0x5f49,0x797d,0x3a9e,0x3bf6,0x5f32,0x1a49,0x3e12,
    0x1cd0,0x1366,0x2e40,0x4944,0x4df2,0x5e14,0x314f,0x4cad,
    0x5422,0x15a1,0x2c3b,0x6032,0x7eb7,0x4230,0x66c4,0x366b,
    0x73da,0x121f,0x798b,0x12e1,0x409d,0x5991,0x0822,0x3ef6,
    0x7049,0x139d,0x5772,0x7bb9,0x0902,0x3699,0x26ca,0x58b0,
    0x4080,0x13e9,0x3cd5,0x6899,0x16c5,0x187e,0x4a80,0x692c,
    0x3cd6,0x5c67,0x60bf,0x5753,0x48cc,0x23c9,0x33ea,0x5db2,
    0x0d66,0x368e,0x54dc,0x422d,0x047e,0x6ad6,0x2f14,0x0fbf,
    0x288f,0x6c69,0x2fff,0x3c61,0x2c49,0x4657,0x75ef,0x7983,
    0x6172,0x1916,0x489c,0x5e9d,0x261e,0x7dd1,0x22cd,0x3a61,
    0x0677,0x494a,0x7f4f,0x0384,0x71f0,0x401d,0x32e6,0x6b72,
    0x0fc9,0x6bcb,0x1953,0x542c,0x5039,0x6be8,0x18d7,0x4402,
    0x5dd5,0x11f4,0x2b0c,0x249e,0x7874,0x2833,0x5f1e,0x0e12,
    0x07cf,0x0035,0x127e,0x2059,0x5fa4,0x4cd4,0x5a9f,0x6ad4,
    0x3a2d,0x0e90,0x01d3,0x46cf,0x0ecc,0x1af4,0x6d22,0x6732,
    0x252a,0x591d,0x19d9,0x37e6,0x0975,0x458f,0x57d3,0x6048,
    0x7b44,0x4087,0x1481,0x5078,0x442b,0x49f7,0x1dc0,0x37e5,
    0x7fbe,0x3a8d,0x7f61,0x16d4,0x2b00,0x1850,0x765f,0x590e,
    0x251f,0x7282,0x0633,0x773b,0x3807,0x0c15,0x5005,0x0c7b,
    0x3bb1,0x39ce,0x4d54,0x5064,0x19da,0x3492,0x6270,0x1d18,
    0x3004,0x486a,0x5c46,0x4ff8,0x6a15,0x6d69,0x513e,0x4c85,
    0x5968,0x4d67,0x182f,0x1f16,0x73d9,0x470e,0x5e73,0x1796,
    0x5876,0x4f68,0x4e57,0x5ed0,0x0a4a,0x3f4a,0x2cf7,0x4ad4,
};

static uint16_t queryCrc16(uint16_t crc16, uint8_t value) {
    return (crc16 >> 8u) ^ CRC_TABLE[(crc16 ^ value) & 0xffu];
}

uint16_t calculateCrc16(const uint8_t *const data, uint32_t size) {
    uint16_t crc16 = 0;
    for (uint32_t index = 0; index < size; index++)
        crc16 = queryCrc16(crc16, data[index]);
    return crc16;
}
```

2.1 设置类功能指令

2.1.1 设备认证登录

⚠ Caution

连接成功后需先进行登录操作,其它指令才能正常交互. (手表必须,戒指可不用登录)

发送Data内容:

1字节	1字节	1字节
CMD	Key	Key flag
0x03	0x02	0x20 0x00 : 改(写)操作 0x10 : 查(读)操作 0x20 : 增操作 0x30 : 删操作

Example:

发送内容: C60100035FC0030220

接收内容: C6110004257503022000

2.1.2 设置设备时区

⚠ Caution

需在时间设置之前,先进行时区设置. 1字节带符号整形 (单位15min) , +为东时区, -为西时区. 比如中国8时区, 8*4 =32.

发送Data内容:

1字节	1字节	1字节	1字节
CMD	Key	Key flag	时区
0x02	0x02	0x00	有符号整形

Example:

发送内容: c6010004616502020020

接收内容: c611000317B0020200

2.1.3 设置设备时间

⚠ Caution

年份为当前年份减去2000; 比如2025-2000 = 25

发送Data内容:

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	年	月	日	时	分	秒
0x02	0x01	0x00	减2000					

Example:

发送内容: c6010009xxxx020100190a1d101732

接收内容: c6110003xxxx020100

2.1.4 获取设备版本信息

发送Data内容: 020410

1字节	1字节	1字节
CMD	Key	Key flag
0x02	0x04	0x10

接收Data内容: 02041000020200f0002801304130353034303200000202

1字节	1字节	1字节	3字节	1字节	2字节	2字节	8字节
CMD	Key	Key flag	版本号	形状	宽	高	型号
0x02	0x04	0x10	0.2.2	0是方, 1是圓	整形	整形	字符串0A050402

Example:

发送内容: c601000340ed020410

接收内容: c6110017xxxx02041000020200f00028013041303530343032

解包内容为: 固件版本号0.2.2, 方形, 分辨率 240x296, 型号0A050402

2.1.5 获取设备电量信息

发送Data内容: 020310

1字节	1字节	1字节
CMD	Key	Key flag
0x02	0x03	0x10

接收Data内容: 0203101e

1字节	1字节	1字节	1字节
CMD	Key	Key flag	电量
0x02	0x03	0x10	整形

Example:

发送内容: c60100034045020310

接收内容: c6110004xxxx0203101e

解包内容为: 电量30,即30%.

2.1.6 设置用户信息

发送Data内容:

1字节	1字节	1字节	1字节	1字节	1字节	4字节	4字节
CMD	Key	Key flag	公英制	性别	年龄	身高(cm)	体重(kg)
0x02	0x06	0x00	0: 公制 1: 英制	0: 女 1: 男	0-100	float	float

Example:

发送内容: c601000exxxx02060000001a00002a4300008242

接收内容: c6110003xxxx020600

2.1.7 定时心率检测设置与获取(仅手表使用)

发送Data内容:

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长
0x02	0x16	0x00 :write 0x10 :read	0: 关 1: 开	0-24	0-60	0-24	0-60	0-255

Example:

获取-发送内容: c601000348fc021610

获取-接收内容: c6110009xxxx02161001000000001e

设置-发送内容: c6010009xxxx0216000100000e001e

设置-接收内容: c611000354bc021600

2.1.8 设置健康数据全天监听间隔

2.1.8.1 心率检测

Note

心率全天检测,只能设置开关与间隔时长(30分钟或60分钟).

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x16	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	30/60

Example:

获取-发送内容: c601000348fc021610

获取-接收内容: c6110009xxxx021610010000173b1e

设置-发送内容: c6010009xxxx021600010000173b1e

设置-接收内容: c611000354bc021600

2.1.8.2 血氧检测

Note

血氧全天检测,只能设置开关, 间隔时长固定60分钟.

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x25	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	60

Example:

获取-发送内容: c601000340c2022510

获取-接收内容: c6110009xxxx022510010000173b1e

设置-发送内容: c6010009xxxx022500010000173b1e

设置-接收内容: c6110003169a022500

2.1.8.3 HRV检测

Note

HRV全天检测,只能设置开关, 间隔时长固定60分钟.

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x6a	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	60

Example:

获取-发送内容: c60100035FD8026a10

获取-接收内容: c6110009xxxx026a10010000173b1e

设置-发送内容: c6010009xxxx026a00010000173b1e

设置-接收内容: c61100033016026a00

2.1.8.4 压力检测

 **Note**

压力全天检测,只能设置开关, 间隔时长固定60分钟.

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x6b	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	60

Example:

获取-发送内容: c60100036784026b10

获取-接收内容: c6110009xxxx026b10010000173b1e

设置-发送内容: c6010009xxxx026b00010000173b1e

设置-接收内容: c6110003153C026b00

2.1.8.5 血糖检测

 **Note**

血糖全天检测,只能设置开关, 间隔时长固定60分钟.

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x6e	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	60

Example:

获取-发送内容: c60100036B0B026e10

获取-接收内容: c6110009xxxx026e10010000173b1e

设置-发送内容: c6010009xxxx026e00010000173b1e

设置-接收内容: c6110003135B026e00

2.1.8.6 血压检测

Note

血压全天检测,只能设置开关, 间隔时长固定60分钟.

1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	开始小时	开始分钟	结束小时	结束分钟	间隔时长(分钟)
0x02	0x7c	0x00 :write 0x10 :read	0: 关 1: 开	00(固定)	00(固定)	23(固定)	59(固定)	60

Example:

获取-发送内容: c60100035fa4027c10

获取-接收内容: c6110009xxxx027c10010000173b1e

设置-发送内容: c6010009300e027c00010000173b1e

设置-接收内容: c6110003xxxx027c00

2.1.9 获取与设置LED亮屏强度(只带屏戒指支持)

发送Data内容:

1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	开关	高度等级
0x02	0x66	0x00 :write 0x10 :read	0: 关 1: 开	1: 微光 2: 柔光 3: 强光

Example:

获取-发送内容: c6010003xxxx026610

获取-接收内容: c6110005xxxx0266100103

设置-发送内容: c6010005xxxx0266100103

设置-接收内容: c6110003xxxx026600

2.1.10 设置控制HID应用类型 (只戒指使用)

(1) 关闭

发送内容: c601000512180264000000

设备回复内容: c6110003765e026400

(2) 打开短视频

发送内容: c6010005492A0264000101

设备回复内容: c6110003765e026400

(3) 打开电子书

发送内容: c601000525540264000201

设备回复内容: c6110003765e026400

(4) 打开音乐

发送内容: c601000501620264000301

设备回复内容: c6110003765e026400

(5) 获取已设置的类型

发送内容: c601000357d2026410

接收内容: c61100051aab0264100000

获取后接收数据部分解析如下表:

0	1	2	3	4
cmd	key	key flag	代表类型: 1 短视频 2 电子书 3 音乐 0 关	开关: 0 关 1 开
0x02	0x64	0x10	0x00~0x03	0x00或0x01

举例发送数据:

```
const data = new Uint8Array([0xC6, 0x01, 0x00, 0x03, 0x57, 0xd2, 0x02, 0x64, 0x00]);

// 转成 ArrayBuffer, 用于 wx.writeBLECharacteristicValue
const buffer = data.buffer;

wx.writeBLECharacteristicValue({
  deviceId,
  serviceId,
  characteristicId,
  value: buffer,
  success: res => console.log('write ok', res),
  fail: err => console.error('write fail', err),
});
```

2.1.11 获取与设置佩戴位置 (只带屏戒指支持)

发送Data内容:

1字节	1字节	1字节	1字节
CMD	Key	Key flag	佩戴位置
0x02	0x68	0x00 :write 0x10 :read	0 : 左手 1 : 右手

Example:

获取-发送内容: c6010003xxxx026810

获取-接收内容: c6110004xxxx02681000

设置-发送内容: c6010004xxxx02680001

设置-接收内容: c6110003xxxx026800

2.1.12 触摸事件通知

① Note

设备触摸事件通知, Protocol flag 为 0x11 (设备主动上报). 触摸操作无论熄屏与否都会上报对应指令, 由App/小程序端定义指令的响应行为.

接收Data内容:

1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	按键类型	触摸类型
0x04	0x06	0x00	1 : 触摸按键(默认) (预留扩展其它按键类型)	1. 单机 2. 双击 3. 三击 4. 长按 5. 甩动

触摸类型说明:

1. 单击 (0x01): 点触一下, 无论熄屏与否, 设备上传单击指令;
2. 双击 (0x02): 双击触摸, 设备上传双击指令;
3. 长按 (0x04): 长按触摸, 设备上传长按指令.

Example (按键类型1, 单击):

接收内容: c6110005xxxx0406000101

无需回复

2.1.13 静息健康数据获取

获取设备静息健康数据, 包含静息心率、静息HRV、静息血氧。

注意: 如果设备一直未佩戴, 或全天定时监测未开启, 则返回值为0。

发送Data内容: 028110

1字节	1字节	1字节
CMD	Key	Key flag
0x02	0x81	0x10

接收Data内容:

1字节	1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	静息心率	静息HRV	静息血氧
0x02	0x81	0x10	整形	整形	整形

Example:

发送内容: c60100032283028110

接收内容: c61100063cc3028110423262

解包内容为: 静息心率66, 静息HRV 50, 静息血氧98.

2.1.14 获取MAC地址

注意: 返回的6字节MAC地址为倒序, 需反转后才是实际MAC地址.

发送Data内容: 020510

1字节	1字节	1字节
CMD	Key	Key flag
0x02	0x05	0x10

接收Data内容:

1字节	1字节	1字节	6字节
CMD	Key	Key flag	MAC地址
0x02	0x05	0x10	6字节地址

Example:

发送内容: c601000307dc020510

接收内容: c61100094a95020510c81b93d119c4

解包内容为: 原始字节 C8:1B:93:D1:19:C4, 反转后实际MAC地址为 C4:19:D1:93:1B:C8

2.1.15 蓝牙名字修改

注意: 名字内容最大24个字节.

发送Data内容:

1字节	1字节	1字节	1字节	N字节
CMD	Key	Key flag	名字长度	名字内容
0x02	0x65	0x00	名字字节数	UTF8编码字符串

Example:

发送内容: c601000a18e902650006e8af95e4baba (名字为"测试人", UTF8编码为 e8af95e4baba, 长度6字节)
接收内容: c61100032b26026500

2.2 控制类功能指令

2.2.1 查找设备

发送内容: C601000413E802340001
接收内容: C61100030170023400

举例发送数据:

```
const data = new Uint8Array([0xC6, 0x01, 0x00, 0x04, 0x13, 0xE8, 0x02, 0x34, 0x00, 0x01]);

// 转成 ArrayBuffer, 用于 wx.writeBLECharacteristicValue
const buffer = data.buffer;

wx.writeBLECharacteristicValue({
  deviceId,
  serviceId,
  characteristicId,
  value: buffer,
  success: res => console.log('write ok', res),
  fail: err => console.error('write fail', err),
});
```

2.2.2 启动与关闭设备健康数据单次检测

⚠ Caution

控制开启戒指心率,血压,血氧,HRV,压力,血糖单次检测; 要提前注意设备是否支持对应数据类型,可在RWfit上查看支持情况.

同一时间只能开启一种健康检测类型, 必须等当前检测结束(收到完成回调)或主动关闭后, 才能启动新的检测类型. 同时开启多种会导致检测异常.

(1) 控制检测开始/结束:

1字节	1字节	1字节	2字节	1字节	1字节
CMD	Key	Key flag	DataType	开始/结束	连续测试
0x06	0x09	0x00	0x0503 : 心率 0x0504 : 血压 0x0509 : 血氧 0x050A : HRV 0x050D : 压力 0x0510 : 血糖	0 : 结束 1 : 开始	0 : 单次测试, 出值后自动停止(默认) 1 : 连续测试, 持续测量不自动停止

 Note

2026-04-13 新增

字段	字节	值/范围	说明
连续测试	1	0 / 1	可选字段, 不传时默认0(单次测试). 0: 出值后自动停止, 1: 持续测量需主动停止

1.心率 Example:

控制开始内容: c60100067efe060900030501
控制开始回复: c61100034092060900

控制结束内容: c60100064279060900030500
控制结束回复: c61100034092060900

2.血压 Example:

控制开始内容: c6010006785c060900040501
控制开始回复: c61100034092060900

控制结束内容: c6010006281b060900040500
控制结束回复: c61100034092060900

3.血氧 Example:

控制开始内容: c6010006017d060900090501
控制开始回复: c61100034092060900

控制结束内容: c60100063007060900090500
控制结束回复: c61100034092060900

4.HRV Example:

控制开始内容: c601000618500609000a0501
控制开始回复: c61100034092060900

控制结束内容: c60100061f690609000a0500

控制结束回复: c61100034092060900

5.压力 Example:

控制开始内容: c60100066e330609000d0501

控制开始回复: c61100034092060900

控制结束内容: c60100061aba0609000d0500

控制结束回复: c61100034092060900

6.血糖 Example:

控制开始内容: c6010006016d060900100501

控制开始回复: c61100034092060900

控制结束内容: c60100063017060900100500

控制结束回复: c61100034092060900

7.心率连续测试 Example:

控制连续开始内容: c6010007xxxx06090003050101

控制连续开始回复: c61100034092060900

控制结束内容: c6010006xxxx060900030500

控制结束回复: c61100034092060900

(2)

心率,血压,血氧,HRV,压力,血糖测试出值后,会主动发送数据,无需回复. 数据格式如下:

1字节	1字节	1字节	6字节
CMD	Key	Key flag	类型数据
0x02	0x24:心率 0x31:血压 0x4E:血氧 0x4F:压力 0x69: HRV 0x6C: 血糖	0x10	HRItem/BloodPressureItem/ BloodOxygenItem/HRVItem/SugarItem/StressItem

注: HRItem, BloodPressureItem, BloodOxygenItem, HRVItem, SugarItem, StressItem定义见[2.3章节](#).

Example:

接收到内容: c6110009xxxx022410.....;

作为ACK回复数据直接返回

(3)

单次测试完成后,会主动通知APP或小程序,需回复

Example:

接收内容: c6010006xxxx060900030500
回复内容: c6110004xxxx06090000

2.2.3 启动与关闭拍照

⚠ Caution
启动拍照功能后,设备可通过手势控制app或小程序自定义相机拍照.

数据格式:

1字节	1字节	1字节	1字节
CMD	Key	Key flag	Type
0x06	0x01	0x00	0.关闭 1.打开 2.拍照

Example:

控制打开内容: c6010004xxxx06010001
控制打开回复: c6110003xxxx060100

控制关闭内容: c6010004xxxx06010000
控制关闭回复: c6110003xxxx060100

2.2.4 控制关机,恢复出厂设置

数据格式:

1字节	1字节	1字节	1字节
CMD	Key	Key flag	Type
0x02	0x22	0x00	1.关机 2.恢复出厂设置

Example:

控制关机内容: c6010004xxxx02220001
控制关机回复: c6110003xxxx022200

控制恢复出厂设置内容: c6010004xxxx02220002

控制恢复出厂设置回复: c6110003xxxx022200

2.2.5 定制健康数据获取(只定制客户使用)

控制检测开始/结束:

1字节	1字节	1字节	2字节	1字节
CMD	Key	Key flag	DataType	开始/结束
0x06	0x09	0x00	0x05ff	0 : 结束 1 : 开始

Example:

控制开始内容: c60100063203060900ff0501

控制开始回复: c61100034092060900

控制结束内容: c60100064E7D060900ff0500

控制结束回复: c61100034092060900

心率,血氧,HRV,血糖测试完成后,会主动发送数据,无需回复. 数据格式如下:

1字节	1字节	1字节	6字节	6字节	6字节	6字节	6字节
CMD	Key	Key flag	心率数据	血氧数据	HRV数据	血糖数据	血压数据
0x02	0x24	0x10	HRItem	BloodOxygenItem	HRVItem	SugarItem	BloodPressureItem

注: HRItem, BloodOxygenItem, HRVItem, SugarItem, BloodPressureItem定义见[2.3章节](#).

Example:

接收到内容: c611001bxxxx022410.....;

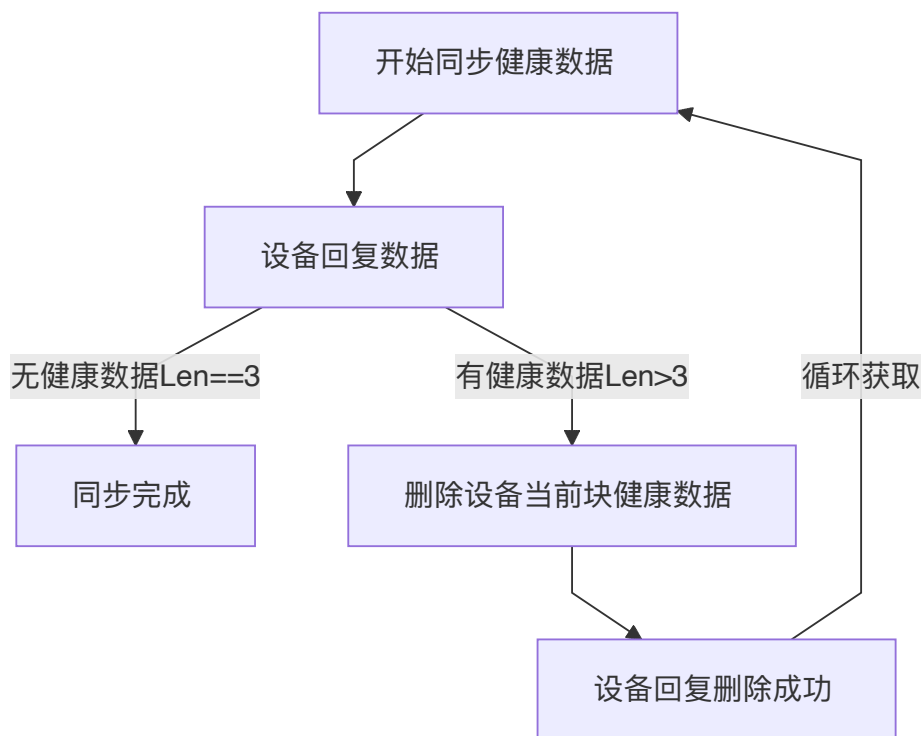
作为ACK回复数据直接返回

2.3 健康数据同步指令

由于戒指每次发送数据都有限制mtu的大小, 所以戒指没办法一次性同步完所有的数据, 需循环获取直到无数据返回才完成.

获取数据----> 有返回数据(Len大于3)----> 删除设备当前块数据 ----> 获取数据----> 有返回数据(Len大于3) ----> 删除设备当前块数据 ----> 无数据(Len等于3) ----> 结束.

注意: 对于每一种健康数据类型都是一样这个流程



以计步数据获取为例：

(1) 发送获取指令：c6010003097b050210

(2) 设备有数据时会返回一块数据：

c6110053xxxx0502103094fd880000007900009bfa000f10e03095010c0000007900009bfa000f10e0309504900000007900009bfa000f10e0309508140000007900009bfa000f10e030950a4b0000010000014a00001fe000

(3) 发送删除指令：c601000319d4050230

(4) 设备返回删除回复：c611000319d4050230

.....

(5) 发送获取指令：c6010003097b050210

(6) 设备无数据时会返回：c6110003097b050210；Len==3无数据返回，计步数据获取完成，结束。

2.3.0 计步今天数据获取

⚠ Caution

计步数据为每小时步数数量；一个StepItem 16个字节，一天24小时，所以一天最大有16*24=384个字节长度，会超过一包的大小，所以也会进行分包发送。第一包里的第一个StepItem为总数量。

数据都为大端模式

发送Data内容：051a10

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x1a	0x10

接收Data内容：

1字节	1字节	1字节	16*N字节
CMD	Key	Key flag	StepItem
0x05	0x02	0x10	N个计步数据

StepItem格式定义:

4字节	1字节	3字节	4字节	4字节
时间戳	模式	步数	卡路里	距离
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	暂无用	历史每小时步数	历史每小时卡路里; 除以10得到卡	历史每小时距离; 除以10000得到米

Example:

发送内容: c6010003097b050210

接收内容:
c6110053xxxx0502103094fd880000007900009bfa000f10e03095010c0000007900009bfa000f10e03095049000000007900009bfa000f10e0309508140000007900009bfa000f10e030950a4b0000010000014a00001fe000

上面接收数据解包后如下:
计步2 time: 815070600 2025-10-29-16:30:00 0 step 121 Cal 39930 Distance 987360
计步2 time: 815071500 2025-10-29-16:45:00 0 step 121 Cal 39930 Distance 987360
计步2 time: 815072400 2025-10-29-17:00:00 0 step 121 Cal 39930 Distance 987360
计步2 time: 815073300 2025-10-29-17:15:00 0 step 121 Cal 39930 Distance 987360
计步2 time: 815073867 2025-10-29-17:24:27 0 step 256 Cal 84480 Distance 2088960

删除计步数据指令:

发送删除: c601000319d4050230

删除返回: c611000319d4050230

2.3.2 睡眠数据获取

⚠ Caution

后一个睡眠状态时间戳减当前状态时间戳为当前状态的持续时间(秒). 一般给17开始状态也算到浅睡里. 如果没有每天都同步, 会有多天数据返回.

发送Data内容: 050510

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x05	0x10

接收Data内容:

1字节	1字节	1字节	7*N字节
CMD	Key	Key flag	SleepItem
0x05	0x05	0x10	N个睡眠状态数据

SleepItem格式定义:

4字节	1字节	1字节	1字节
时间戳	睡眠状态		
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	17: 睡眠开始 34: 睡眠结束 1: 深睡 2: 浅睡 3. 清醒 4. REM眼动	暂无用	暂无用

Example:

发送内容: c6010003600e050510

接收内容: c6110003xxxx050510

删除睡眠数据指令:

发送删除: c60100031d5e050530

删除返回: c61100031d5e050530

2.3.3 心率数据获取

发送Data内容: 050310

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x03	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	HRItem
0x05	0x03	0x10	N个心率数据

HRItem格式定义:

4字节	1字节	1字节
时间戳	心率值	
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数	暂无用

Example:

发送内容: c60100030a50050310

接收内容: c6110003xxxx050310

删除心率数据指令:

发送删除: c6010003381d050330

删除返回: c6110003381d050330

2.3.4 血氧数据获取

发送Data内容: 050910

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x09	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	BloodOxygenItem
0x05	0x09	0x10	N个血氧数据

BloodOxygenItem格式定义:

4字节	1字节	1字节
时间戳	血氧值	
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数	暂无用

Example:

发送内容: c6010003386d050910

接收内容: c6110003xxxx050910

删除血氧数据指令:

发送删除: c60100030a20050930

删除返回: c61100030a20050930

2.3.4 压力数据获取

发送Data内容: 050D10

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x0D	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	StressItem
0x05	0x0D	0x10	N个压力数据

StressItem格式定义:

4字节	1字节	1字节
时间戳	压力值	
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数	暂无用

Example:

发送内容: c60100031346050d10

接收内容: c6110003xxxx050d10

删除压力数据指令:

发送删除: c601000313c9050d30

删除返回: c611000313c9050d30

2.3.5 HRV数据获取

发送Data内容: 050A10

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x0A	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	HRVItem
0x05	0x0A	0x10	N个HRV数据

HRVItem格式定义:

4字节	1字节	1字节
时间戳	HRV值	
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数	暂无用

Example:

发送内容: c6010003782b050a10

接收内容: c6110003xxxx050a10

删除HRV数据指令:

发送删除: c60100032c16050a30

删除返回: c61100032c16050a30

2.3.6 血糖数据获取

发送Data内容: 051010

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x10	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	SugarItem
0x05	0x10	0x10	N个血糖数据

SugarItem格式定义:

4字节	2字节
时间戳	血糖值
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数, 结果需除以10,保留一位小数.

Example:

发送内容: c60100032ef9051010

接收内容: c6110003xxxx051010

删除血糖数据指令:

发送删除: c60100031a8b051030

删除返回: c61100031a8b051030

2.3.7 血压数据获取

发送Data内容: 050410

1字节	1字节	1字节
CMD	Key	Key flag
0x05	0x04	0x10

接收Data内容:

1字节	1字节	1字节	6*N字节
CMD	Key	Key flag	BloodPressureItem
0x05	0x10	0x10	N个血压数据

BloodPressureItem格式定义:

4字节	1字节	1字节
时间戳	收缩压	舒张压
每小时时间戳 减了2000年的时间戳, 计算时加946684800u	整数	整数

Example:

发送内容: c6010003xxxx050410

接收内容: c6110003xxxx050410

删除血压数据指令:

发送删除: c6010003xxxx050430

删除返回: c6110003xxxx050430

3. 固件升级 (只戒指使用)

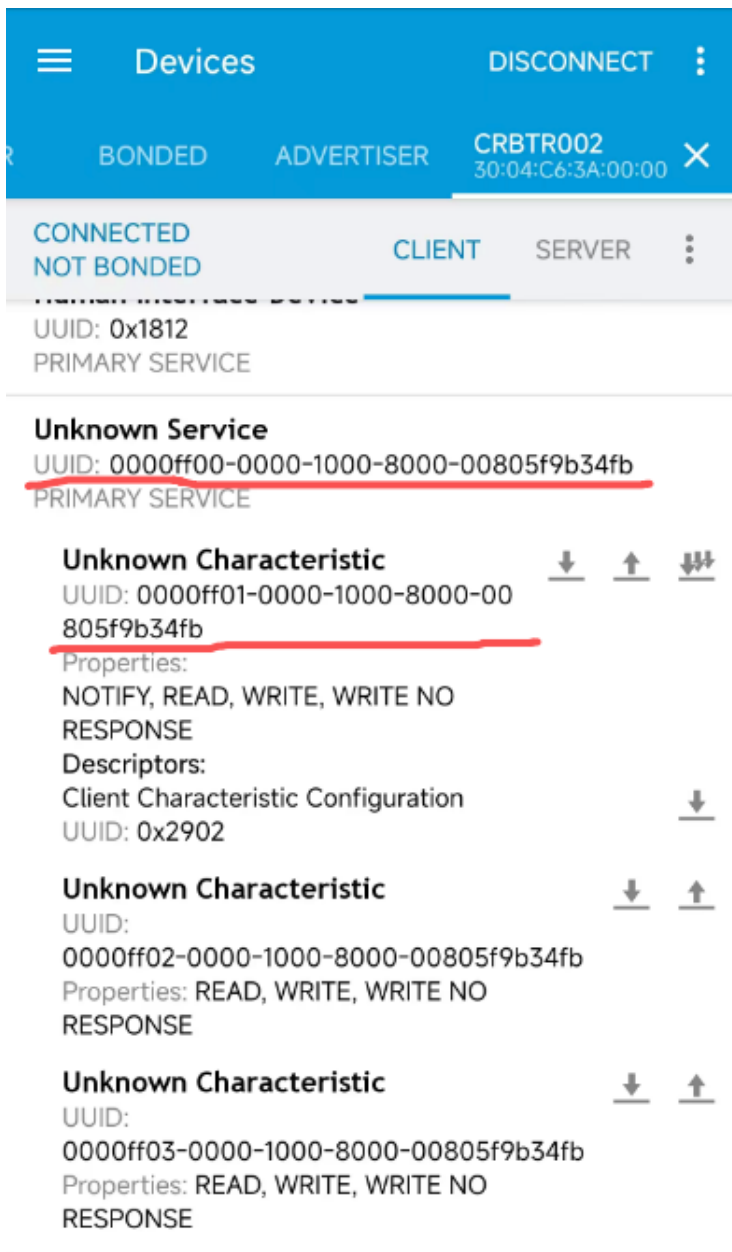
3.1 升级前注意事项

Important

- 提供设备型号与版本号给到方案商获取升级包bin文件.
- 升级时判断电量大于40%以上才进行升级,防止升级中出现没电情况.
- 所有数据都是采用小端模式.

3.2 蓝牙升级服务与特征UUID

固件升级使用 Service: FF00, Characteristic: FF01(支持 Notify, Read, Write, Write No Response) 进行升级时数据收发.



3.3 指令格式与协议流程

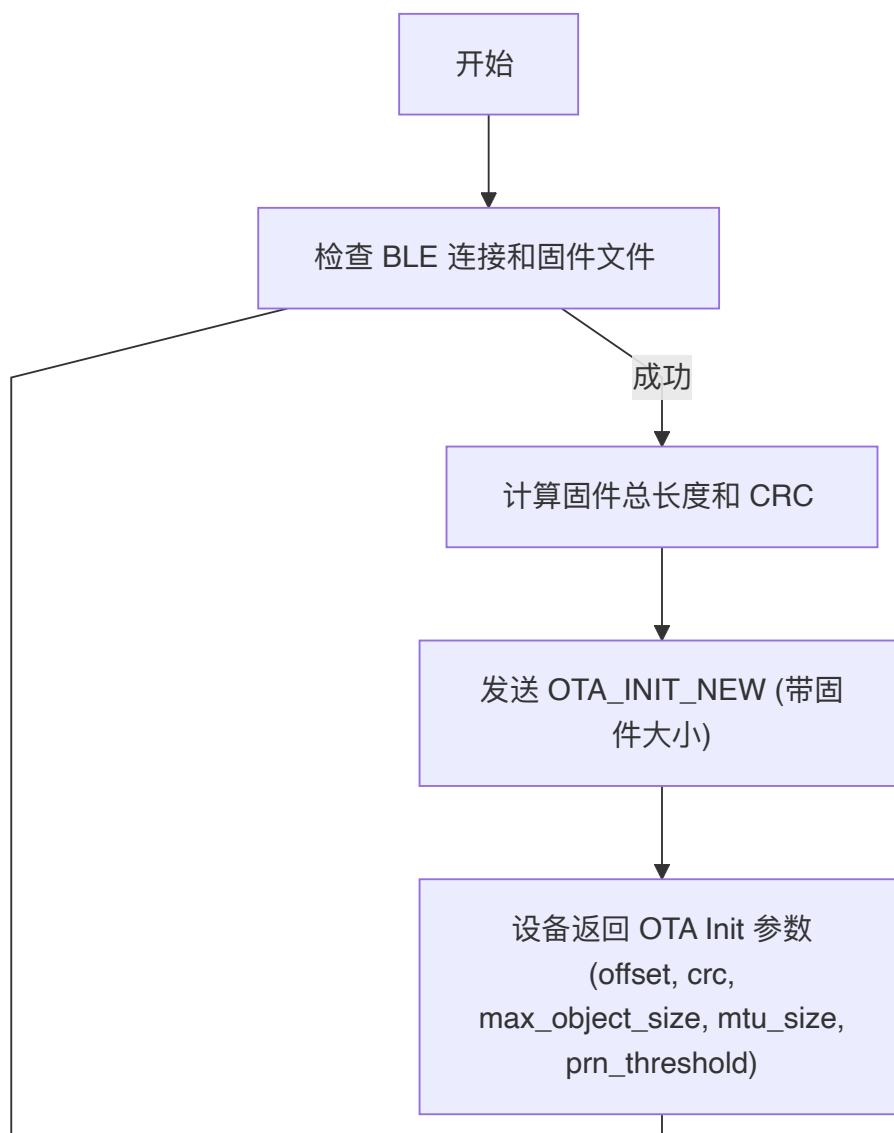
3.3.1 相关变量术语说明

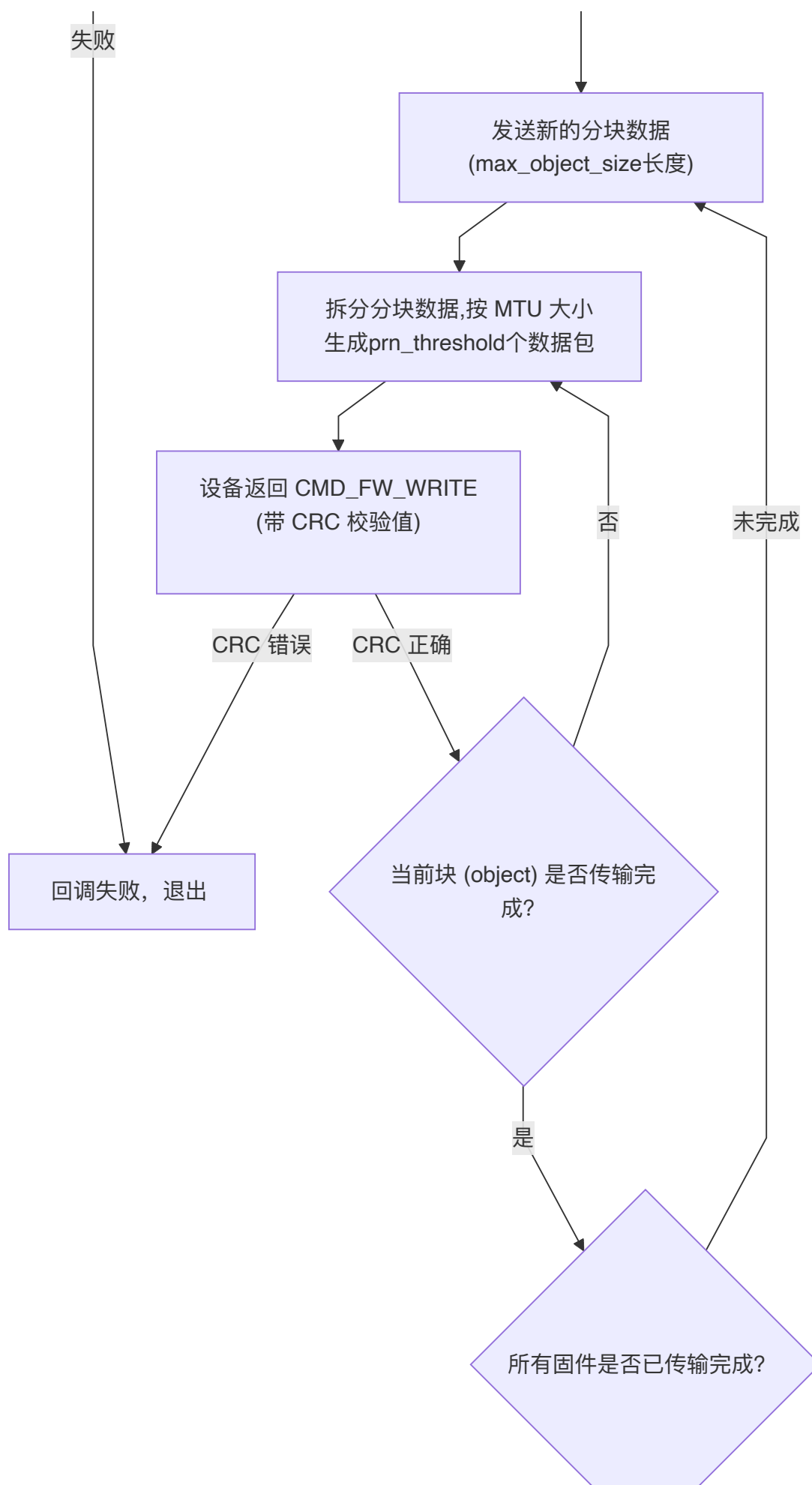
- max_object_size : 分块大小, 一个bin文件上百KB,会被分成多个块发送到设备,比如一个块4096字节;
- mtu_size : 一次能发送的字节数, 就是 wx.writeBLECharacteristicValue 具体发送的内容长度; 块会被分成多个mtu_size 去发送.
- prn_threshold: PRN 阈值, 作用是 控制设备在接收多少个数据包之后给 App 返回一次确认.

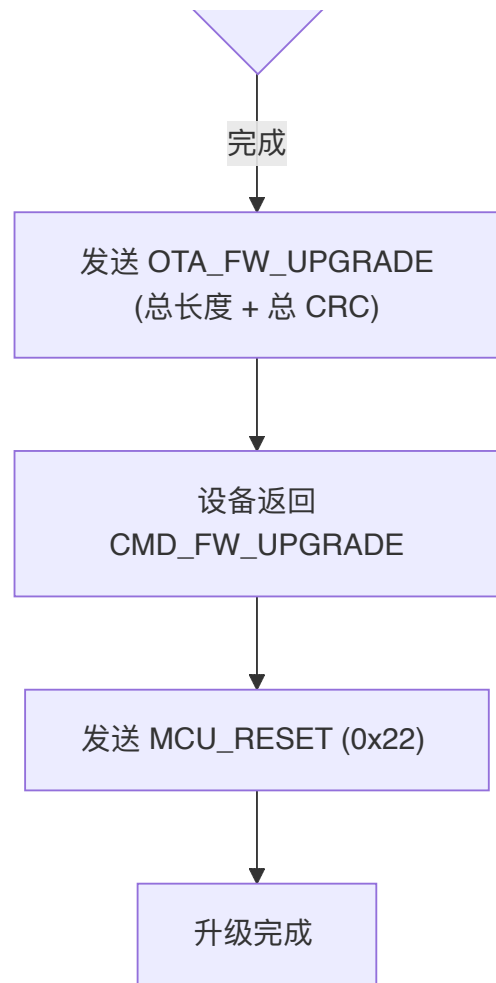
3.3.2 升级整体逻辑

1. App 发送 OTA 初始化请求 (包含固件大小) , 设备返回参数 (MTU、分块大小、PRN 阈值等) 。
2. 分块传输固件数据:
 - App 每次创建一个分块对象 (Object, 典型大小 4KB) ,

- 按 MTU 拆包发送数据，累计 CRC 校验。
3. 设备确认 **CRC 正确**，App 再进入下一个对象传输。
 4. 所有数据传输完成 → App 发送 **升级指令**（带总大小、CRC）。
 5. 设备执行升级，最后 **MCU Reset**，升级完成。







3.3.3 升级指令格式

3.3.3.1 文件数据CRC校验

校验算法:

```
- (uint32_t)getSourceFileCRC:(NSData *)binData {
    uint32_t tCRC = 0;
    const uint8_t *bytes = (const uint8_t *)[binData bytes];
    NSUInteger length = [binData length];

    for (NSUInteger i = 0; i < length; i++) {
        uint32_t CC = (uint32_t)bytes[i];
        CC ^= 0x000000FF;
        tCRC += CC;
        tCRC = tCRC & 0x0000FFFF;
    }
    return tCRC;
}
```

注意: 上面是OC语言实现可参考.

3.3.3.2 获取设备升级初始化参数信息 OTA_Init_New

发送:

1字节	4字节
CMD_FW_OTA_INIT_NEW (0x27)	FWLength
指令类型	固件长度

接收:

1字节	1字节	1字节	2字节	2字节	2字节	4字节	2字节	2字节
0x0e		0x27				max_object_size	mtu_size	prn_threshold
指令类型	忽略	指令类型	忽略	文件偏移位置	CRC	分块大小	MTU	PRN 阈值

例子:

```
发送: {length = 5, bytes = 0x27b0e70200}
接收: {length = 18, bytes = 0x0e1027000100000000001000006400010001}
```

 Important

此条指令发送后, 接收需使用 `wx.readBLECharacteristicValue(Object object)` 不会主动Notfiy. 建议延迟0.3s调用.
文件偏移 都会为0.

3.3.3.3 发送分块开始信息 OTACreate

发送:

1字节	4字节	4字节
CMD_FW_OBJECT_CREATE (0x25)	index	max_object_size
指令类型	文件偏移位置	分块大小

接收:

1字节	1字节	1字节
0x25	0x00	0x00
指令类型	忽略	忽略

例子:

```
发送: {length = 9, bytes = 0x250000000000100000}
接收: {length = 3, bytes = 0x250000}
```

3.3.3.4 发送分块数据

把分块数据max_object_size, 按mtu_size大小一包一包发送, 发送prn_threshold包后,会返回设备侧整个已接收到的bin文件数据部分的CRC. CRC校验一致 继续 循环发送, 直到把当前分块数据发送完成. 再继续下一个分块数据发送.

最后一包如果不满mtu_size按剩余实际大小发送

发送:

<=mtu_size字节
bin文件Uint8Array
内容

接收(当发送完prn_threshold组数组后会返回):

1字节	2字节
0x17	CRC
指令类型	设备侧已接收到bin文件数据部分的CRC

例子:

```
//当prn_threshold == 1时,发送一包就返回数据
发送: {length = 100, bytes = 0x88780020 d5000010 dd000010 81530010 ... f9a90010 65a80010 }
接收: {length = 3, bytes = 0x173b0f}
```

3.3.3.5 发送结束信息 OTAFWupgrade

发送:

1字节	4字节	2字节
CMD_FW_UPGRADE (0x18)	FWLength	CRC
指令类型	文件总长度	文件校验码

接收:

1字节	1字节	1字节
0x18	0x00	0x00
指令类型	忽略	忽略

例子:

```
发送: {length = 7, bytes = 0x18b0e702002db9}
接收: {length = 3, bytes = 0x180000}
```

3.3.3.6 发送重置信息 OTA_FW_MCU_RESET

Important

此指令发送后,就代表整个流程已完成,可提示升级完成, 等待设备重启.

发送:

1字节	1字节
CMD_FW_MCU_RESET (0x22)	0x00
指令类型	固定0x00

无返回

例子:

```
发送: {length = 2, bytes = 0x2200}
```

指令枚举类型

Caution

指令类型只使用了一部分，就是上面有列举的指令.

```
typedef enum : UInt8 {
    CMD_FW_INIT = 0x10,
    CMD_FW_SET_ADDRESS = 0x11,
    CMD_FW_ERASE = 0x16,
    CMD_FW_WRITE = 0x17,
    CMD_FW_UPGRADE = 0x18,
    CMD_FW_SECTOR_ERASE = 0x19,
    CMD_FW_WRITE_SET = 0x20,
    CMD_FW_CONNECTION_UPDATE = 0x21,
    CMD_FW_MCU_RESET = 0x22,
    CMD_FW_GET_ATT_MTU = 0x24,
    CMD_FW_OBJECT_CREATE = 0x25,
    CMD_FW_EXECUTE = 0x26,
    CMD_FW_OTA_INIT_NEW = 0x27,
    EVT_COMMAND_COMPLETE = 0x0E
} PXI_CMD_FW;
```

4 Sensor原始数据获取

Important

此功能只有部分设备型号版本才支持,不能启动或收不到数据可咨询厂家.

4.1 PPG与ACC原始数据启动与获取

(1) 控制检测开始/结束:

1字节	1字节	1字节	1字节	1字节
CMD	Key	Key flag	outputType	sensorType
0x02	0xFA	0x00	0x01 : 开启Sensor输出 0x02 : 关闭Sensor输出	按位组合值: 第0位: ACC(0x01) 第1位: PPG绿光(0x02) 第2位: PPG Red红光(0x04) 第3位: IR红外(0x08) 当前支持组合: 1 : ACC 2 : 绿光 3 : 绿光+ACC 4 : 红光 5 : 红光+ACC 10 : 绿光+红外 11 : 绿光+ACC+红外 12 : 红光+红外 13 : 红光+ACC+红外

1. ACC控制 Example:

控制开始内容: c601000557e902fa000101

控制开始回复: c6110003594a02FA00

控制结束内容: c6010005047302fa000201

控制结束回复: c6110003594a02FA00

2. 绿光控制 Example:

控制开始内容: c601000545b502fa000102

控制开始回复: c6110003594a02FA00

控制结束内容: c60100050fb202fa000202

控制结束回复: c6110003594a02FA00

3. 绿光+ACC控制 Example:

控制开始内容: c6010005094f02fa000103

控制开始回复: c6110003594a02FA00

控制结束内容: c60100052f1902fa000203

控制结束回复: c6110003594a02FA00

4. 红光控制 Example:

控制开始内容: c601000537dc02fa000104

控制开始回复: c6110003594a02FA00

控制结束内容: c6010005368302fa000204

控制结束回复: c6110003594a02FA00

5. 红光+ACC控制 Example:

控制开始内容: c601000519e302fa000105

控制开始回复: c6110003594a02FA00

控制结束内容: c60100050d6b02fa000205

控制结束回复: c6110003594a02FA00

6. 绿光+红外控制 Example:

控制开始内容: c601000549cd02fa00010a

控制开始回复: c6110003594a02FA00

控制结束内容: c6010005798e02fa00020a

控制结束回复: c6110003594a02FA00

7. 绿光+ACC+红外控制 Example:

控制开始内容: c6010005441102fa00010b

控制开始回复: c6110003594a02FA00

控制结束内容: c601000575e202fa00020b

控制结束回复: c6110003594a02FA00

8. 红光+红外控制 Example:

控制开始内容: c6010005504202fa00010c

控制开始回复: c6110003594a02FA00

控制结束内容: c60100056c6402fa00020c

控制结束回复: c6110003594a02FA00

9. 红光+ACC+红外控制 Example:

控制开始内容: c601000514bb02fa00010d

控制开始回复: c6110003594a02FA00

控制结束内容: c6010005288202fa00020d

控制结束回复: c6110003594a02FA00

(2) PPG(绿光/红光/红外)出值后,会主动发送数据,无需回复.** 数据格式如下:

1字节	1字节	1字节	1字节	1字节	N字节
CMD	Key	Key flag	sensorType	type	数据内容
0x02	0xfb	0x00	0x01	见下	根据type解析, 均为小端格式

其中 `type` 定义如下:

- `type=0`: 时间戳, 4字节, 小端.
- `type=1`: 绿光(PPG)数据, `int32`.
- `type=3`: 红光(PPG Red)数据, `int32`.
- `type=4`: 红外(IR)数据, `int32`.

注意:

- 该数据包采用**连续字节流**格式。在 `sensorType` 字段之后, 紧随其后还有 **1字节 `type`** 字段.
- 当 `type=0` 时, 后续数据固定为 **4字节时间戳**, 采用 **Little-Endian(小端模式, 低字节在前)**.
- 当 `type=1/3/4` 时, 后续数据为对应类型的 **int32** 数据, 采用 **Little-Endian(小端模式, 低字节在前)**.
- 若一次上报包含多个同类型采样点, 则按时间先后顺序连续排列, 中间无填充; 开发者可根据数据总长度按对应单个数据宽度循环解析.

(3) ACC出值后, 会主动发送数据, 无需回复.** 数据格式如下:

1字节	1字节	1字节	1字节	2字节	2字节	2字节	...
CMD	Key	Key flag	sensorType	ACC X值, 小端	ACC Y值, 小端	ACC Z值, 小端	...
0x02	0xfb	0x00	0x02	X(int16)	Y(int16)	Z(int16)	...

注意: 该数据包采用**连续字节流**格式。在 `sensorType` 字段 (Offset 3) 之后, 紧随其后的 6 个字节为三轴加速度原始数据。数据按 **X、Y、Z** 的顺序排列, 每个轴占用 **2 字节 (int16)**, 且均遵循 **Little-Endian (小端模式, 低字节在前)**。开发者需通过偏移量累加的方式依次提取。

常见问题

1. 如何判断BLE蓝牙发送与接收都正常?

在按1.1与1.2连接成功, 特征设置成功后, 可使用2.2.1查找指令进行验证, 发送成功设备会亮屏或亮灯, 同时会接收到设备的回复数据.

2. 如何判断数据部分CRC计算正确?

在按第2章部分CRC算法转成小程序代码 计算后, 可对比2.2.1等已算出的CRC进行对比而判断是否正确. 有些加了xxxx的代表CRC值未填写.

- 2.2.1 查找指令发送内容: `C601000413E802340001` `13E8`即为计算的CRC.
- 2.2.2 控制心率单次测试开始内容: `c60100067efe060900030501` `7efe`即为计算的CRC.

3. 如何获取健康数据?

健康数据分为二种: 一种为单次实时测试数据, 另一种为全天历史监测数据.

(1) 单次实时测试数据: 需小程序指令进行启动与结束,测量结果实时返回, 最后一条结果也会保存进历史数据; 见[2.2.2章节](#)指令.

(2) 全天历史监测数据: 全天按间隔时间30分钟或60分钟设备自动进行测量,可通过[2.1.8章节](#)开启与设置间隔时间; 且通过[2.3章节](#)同步历史数据.

4. 可以加一个指令队列,防止业务过多时指令错乱; 发送超时等; 按业务情况添加.

修订记录

更新日期	内容描述
20250902	1. 添加戒指OTA协议流程;
20250904	1. 加HID控制指令内容; 2. 添加部分小程序开发说明;
20251029	1. 加登录,时间,用户信息,心率检测设置指令; 2. 加健康数据同步指令; 3. 加CRC16算法,调整一些固定指令错误.
20251030	1. aizen定制功能定义. 2. 完善指令内容;
20251201	1. 更新固件支持小程序版本要求. 2. 加常见问题列表.
20260306	1. 加Sensor PPG, ACC数据获取.
20260310	1. 加血压检测.
20260314	1. 更新Sensor原始数据 <code>sensorType</code> 说明与Example. 2. 补充PPG原始数据 <code>type</code> 中时间戳、PPG Red、IR说明.
20260330	1. 加触摸事件通知指令(2.1.12). 2. 加静息健康数据获取指令(2.1.13).
20260401	1. 加获取MAC地址指令(2.1.14).
20260407	1. 加蓝牙名字修改指令(2.1.15).
20260413	1. 2.2.2 单次健康检测指令新增连续测试字节,支持持续测量模式.